CYBERCAMP.AI / SECURITY WORKSHOP

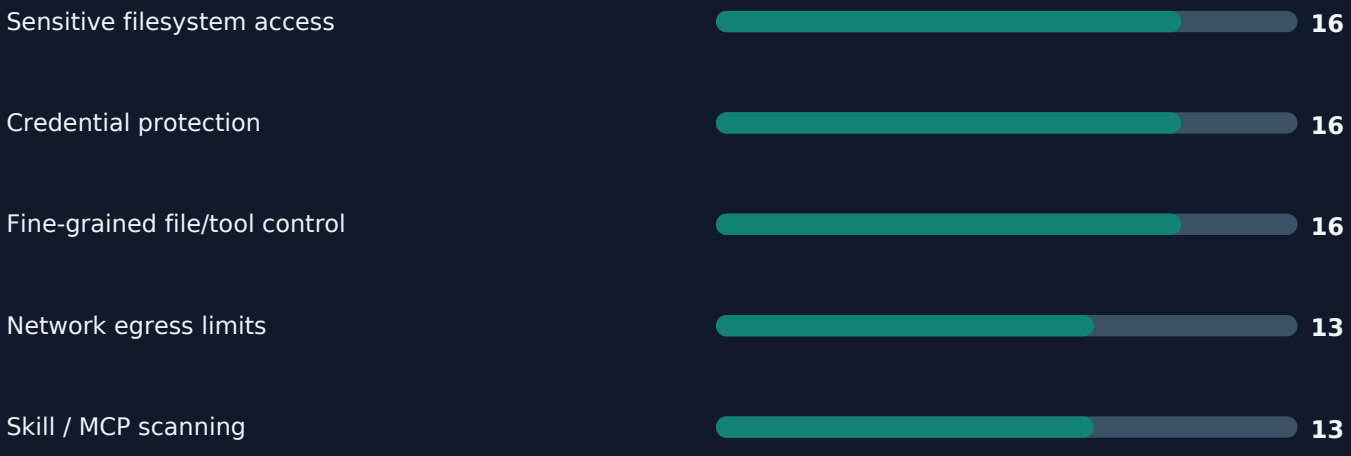
20 Minutes to a More Secure Agent

Participant-driven tool and configuration map | 06 November 2026

SURVEY SNAPSHOT

20	13	3	2	2
responses	Claude Code only	Hermes only	Codex only	mixed agents

WHAT PARTICIPANTS ASKED FOR



DESIGN PRINCIPLE: Native permissions first. Add nono only for an uncovered whole-process boundary; use OpenShell when mixed/custom agents require a shared enforced policy and credential boundary.

7 cohorts / 7 dedicated illustrated-setting pages / anonymized survey mapping

A Claude Code / macOS

Filesystem protection, credential handling, tool approvals, skill scanning; optional egress.

7 participants

Survey responses: 8, 9, 10, 12, 15, 16, 18

NATIVE APPROACH

Claude Code sandbox (Seatbelt), permission rules, workspace-only file tools and credential scrubbing.

CONFIG FILES / ENTRY POINTS

~/.claude/settings.json | .claude/settings.json | .mcp.json

WHEN TO ADD NONO OR OPENSHELL

No nono/OpenShell for the baseline. Add nono ONLY to sandbox the entire Claude process, including local MCP/hook processes; its key proxy is not universal OAuth brokering.

ILLUSTRATIVE SETTINGS

~/.claude/settings.json

```
{
  "permissions": {
    "deny": ["Read(~/.ssh/**)", "Read(**/.env)", "Edit(**/.env)"],
    "blockReadsOutsideWorkingDirectories": true,
    "blockWritesOutsideWorkingDirectories": true
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false,
    "filesystem": {"denyRead": ["~/.ssh", "~/.aws"]},
    "network": {"allowedDomains": ["github.com"]},
    "credentials": {"envVars": [
      {"name": "GITHUB_TOKEN", "mode": "deny"}
    ]}
  }
}
```

VALIDATE IN THE LAB

- Use /sandbox to confirm Seatbelt enforcement; have Claude run a blocked read and curl to a non-allowlisted domain.
- Scan skills: skill-scanner scan ./skill --use-llm; scan MCP configs: uvx snyk-agent-scan@latest (consent before contacting servers).

IMPORTANT BOUNDARY

Sandbox settings govern Bash subprocesses, not built-in WebFetch, local MCP servers or hooks. Read/Edit rules govern file tools. A user-typed ! command is typically outside the native Bash sandbox.

Reference: code.claude.com/docs/en/sandboxing; code.claude.com/docs/en/permissions

B Claude Code / Linux

Workspace-only edits, blocked credential reads and restricted shell egress on Ubuntu/Fedora.

4 participants

Survey responses: 2, 5, 14, 20

NATIVE APPROACH

Native sandbox (bubblewrap + socat), native permissions, credential deny/mask settings.

CONFIG FILES / ENTRY POINTS

~/.claude/settings.json | .claude/settings.json

WHEN TO ADD NONO OR OPENSHELL

No nono needed for the native Bash-focused lab. Use nono only for a full-process OS boundary covering local MCP servers and other unsandboxed child processes.

ILLUSTRATIVE SETTINGS

~/.claude/settings.json

```
{
  "permissions": {
    "deny": ["Read(~/.ssh/**)", "Read(**/.env)"],
    "blockReadsOutsideWorkingDirectories": true,
    "blockWritesOutsideWorkingDirectories": true
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false,
    "filesystem": {"denyRead": ["~/.ssh", "~/.aws"]},
    "network": {"allowedDomains": ["github.com"]}
  }
}
```

VALIDATE IN THE LAB

- Ubuntu/Debian: sudo apt install bubblewrap socat; Fedora: sudo dnf install bubblewrap socat.
- Use /sandbox, then ask Claude to attempt: touch ~/sandbox-probe and curl --noproxy '*' https://example.com.

IMPORTANT BOUNDARY

AppArmor/user-namespace policy may block bubblewrap; failIfUnavailable avoids an unintentional unsandboxed fallback. A blocked file tool is not proof that arbitrary subprocesses are blocked.

Reference: code.claude.com/docs/en/sandboxing; code.claude.com/docs/en/permissions

C Claude Code / Windows + WSL2

Linux-native Claude sandbox; keep Windows credentials and host filesystem outside the agent.

2 participants

Survey responses: 6, 17

NATIVE APPROACH

Claude Code sandbox INSIDE WSL2, bubblewrap/socat, permission rules, hardened WSL boundary.

CONFIG FILES / ENTRY POINTS

WSL: ~/.claude/settings.json | /etc/wsl.conf (WSL automount/interop)

WHEN TO ADD NONO OR OPENSHELL

No OpenShell by default. Add nono on WSL only after checking Landlock availability, and only if whole-process isolation is required.

ILLUSTRATIVE SETTINGS

Inside WSL: ~/.claude/settings.json

```
{
  "permissions": {
    "deny": ["Read(~/.ssh/**)", "Read(**/.env)"],
    "blockReadsOutsideWorkingDirectories": true
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false,
    "filesystem": {"denyRead": ["~/.ssh", "/mnt/c"]},
    "network": {"allowedDomains": ["github.com"]}
  }
}
```

VALIDATE IN THE LAB

- PowerShell: wsl -l -v; inside WSL: sudo apt install bubblewrap socat.
- Inspect /etc/wsl.conf for Windows drive automount and interop; restart WSL after changes.
- Test blocked /mnt/c reads; scan skills and MCP configs (Cisco Skill Scanner / Snyk Agent Scan).

IMPORTANT BOUNDARY

WSL2 is required. Windows interop through Unix sockets and mounted host drives are separate paths to review; optionally install the seccomp filter. Do not run Claude natively on Windows for this sandbox demo.

Reference: code.claude.com/docs/en/sandboxing; Microsoft WSL configuration docs

D Codex / macOS

File/tool permissions, outbound host allowlisting, and skill security scanning.

2 participants

Survey responses: 4, 19

NATIVE APPROACH

Codex beta permission profiles, network proxy, approval policy and optional executable rules.

CONFIG FILES / ENTRY POINTS

~/.codex/config.toml | ~/.codex/rules/*.rules | AGENTS.md

WHEN TO ADD NONO OR OPENSHELL

No nono/OpenShell needed: native Codex profiles now express both filesystem read/write/deny and domain allowlisting. Add external isolation only for non-Codex subprocess trust boundaries.

ILLUSTRATIVE SETTINGS

~/.codex/config.toml (beta permissions profile)

```
default_permissions = "workshop"
approval_policy = "on-request"

[features]
network_proxy = true

[permissions.workshop]
extends = ":workspace"

[permissions.workshop.filesystem]
":root" = "deny"
":minimal" = "read"

[permissions.workshop.filesystem.":workspace_roots"]
"." = "write"
"**/*.env" = "deny"

[permissions.workshop.network]
enabled = true
[permissions.workshop.network.domains]
"api.github.com" = "allow"
```

VALIDATE IN THE LAB

- Do not mix default_permissions with sandbox_mode or sandbox_workspace_write: the older keys override permission profiles.
- Verify the installed Codex version supports beta profiles; test a denied .env read and unknown-domain request.
- Use Cisco Skill Scanner for skills; Snyk Agent Scan to inspect MCP server configuration with consent.

IMPORTANT BOUNDARY

Network domain filtering requires BOTH network.enabled and features.network_proxy. Profiles are beta; the sample is NOT compatible with older Codex versions. MCP OAuth credentials are a separate consideration.

Reference: learn.chatgpt.com/docs/permissions; learn.chatgpt.com/docs/config-file/config-basic

E Hermes / Linux or WSL2

Secure files, network, credentials and skills while using native Hermes sandbox backends.

2 participants

Survey responses: 1, 3

NATIVE APPROACH

Hermes Docker terminal backend, approvals, toolset controls, Skills Guard, write approval and native iron-proxy egress.

CONFIG FILES / ENTRY POINTS

~/.hermes/config.yaml | ~/.hermes/.env | ~/.hermes/proxy/proxy.yaml

WHEN TO ADD NONO OR OPENSHELL

No nono or OpenShell for the normal lab: Hermes already provides isolated command execution and a dedicated proxy. Use stronger external isolation only if the Hermes host process itself is untrusted.

ILLUSTRATIVE SETTINGS

~/.hermes/config.yaml

```
terminal:
  backend: docker
  docker_forward_env: []
  container_persistent: false
approvals:
  mode: manual
  deny:
    - "git push --force*"
security:
  redact_secrets: true
  website_blocklist:
    enabled: true
    domains: ["*.internal.example"]
skills:
  write_approval: true
proxy:
  enabled: true
  enforce_on_docker: true
  extra_allowed_hosts: []
```

VALIDATE IN THE LAB

- Run: hermes egress install; hermes egress setup; hermes egress start; hermes egress status.
- Review toolsets with hermes tools; use Cisco Skill Scanner for skills and Snyk Agent Scan for MCP integrations.
- From a Docker sandbox, test outbound to an unapproved host and inspect proxy decisions.

IMPORTANT BOUNDARY

Hermes write-file guards do NOT constrain arbitrary shell commands. Docker isolation does. The native proxy handles known provider-key schemes, not every OAuth/SigV4 credential; enforce network egress at the network boundary.

Reference: hermes-agent.nousresearch.com/docs/user-guide/security; .../egress/iron-proxy

F Hermes / macOS host, Ubuntu agent

Credentials and tool permissions for Hermes running in an Ubuntu environment.

1 participant

Survey responses: 11

NATIVE APPROACH

Hermes Docker/remote terminal backend and tool ACL; host-held secrets + Hermes egress proxy for Docker.

CONFIG FILES / ENTRY POINTS

Ubuntu: ~/.hermes/config.yaml | ~/.hermes/proxy/proxy.yaml | ~/.hermes/.env

WHEN TO ADD NONO OR OPENSHELL

No nono for the Docker backend. OpenShell is disproportionate for a simple Hermes setup; consider it only for centralized multi-agent credentials/policies.

ILLUSTRATIVE SETTINGS

Ubuntu Hermes runtime: ~/.hermes/config.yaml

```
terminal:
  backend: docker
  docker_forward_env: []
  container_persistent: false
approvals:
  mode: manual
  mcp_reload_confirm: true
security:
  redact_secrets: true
skills:
  write_approval: true
proxy:
  enabled: true
  enforce_on_docker: true
  extra_allowed_hosts: []
```

VALIDATE IN THE LAB

- Run hermes config check, inspect the active runtime with hermes tools.
- Configure hermes egress setup/start and verify the sandbox contains proxy tokens, not provider keys.
- Confirm which macOS or Ubuntu host actually owns the credentials and which runtime runs the agent.

IMPORTANT BOUNDARY

A macOS host with an Ubuntu guest is not the same as macOS-local Hermes. Run setup and validation where Hermes actually executes; a proxy on another host does not protect a guest automatically.

Reference: hermes-agent.nousresearch.com/docs/user-guide/configuration; .../egress/iron-proxy

G Mixed agents / advanced isolation

Claude + Codex on Windows/Linux, and Claude + Hermes + custom agents on Ubuntu.

2 participants

Survey responses: 7, 13

NATIVE APPROACH

Begin with each agent's native policy: Claude settings.json; Codex config.toml; Hermes config.yaml. Compare gaps.

CONFIG FILES / ENTRY POINTS

Agent-native configs + optionally OpenShell ./policy.yaml

WHEN TO ADD NONO OR OPENSHELL

OpenShell JUSTIFIED when one enforceable boundary must cover mixed/custom agent processes, outbound APIs and brokered credentials. Nono is the lighter local full-process option.

ILLUSTRATIVE SETTINGS

Optional: ./openshell-policy.yaml (advanced track)

```
version: 1
filesystem_policy:
  include_workdir: true
  read_only:
    - /usr
    - /lib
    - /etc
  read_write:
    - /tmp
landlock:
  compatibility: best_effort
network_policies:
  github_readonly:
    endpoints:
      - host: api.github.com
        port: 443
        protocol: rest
        enforcement: enforce
        access: read-only
    binaries:
      - path: /usr/bin/curl
```

VALIDATE IN THE LAB

- Native baseline first: compare what a shell, built-in file tool and MCP tool can reach.
- Advanced: openshell sandbox create --policy ./openshell-policy.yaml -- claude (if installed/configured).
- Demonstrate denied network; scan skills/MCPs; attach credential providers separately with explicit destination bindings.

IMPORTANT BOUNDARY

An egress allowlist is NOT a credential policy. OpenShell provider attachment/binding must be configured separately. For Claude/Codex on mixed machines, keep each native policy even when adding an outer sandbox.

Reference: docs.nvidia.com/openshell/latest/reference/policy-schema; .../sandboxes/policies